

AHM

ECOMMERCE SYSTEM

A full-stack, role-based ecommerce platform for luxury fragrances & cosmetics — built with Node.js, Express, MySQL, and Vanilla JS following the Model-View-Controller (MVC) architecture.

GROUP MEMBERS

Abdullah

24K-0859

Hammad

24K-0544

Abdul Majid

24K-0895

1. Abstract

The **AHM Ecommerce System** is a comprehensive full-stack web application designed to provide a premium online shopping experience for luxury fragrances and cosmetics. Built using **Node.js** and **Express** for the backend and **MySQL** for relational data management, the system features a robust security layer using **JWT authentication** and **bcrypt** password hashing. The application follows a **Model-View-Controller (MVC)** architecture, ensuring a clean separation of concerns and scalability.

2. Introduction

2.1 Background

In the modern digital era, the beauty and fragrance industry has seen a massive shift toward online retail. Consumers expect fast, secure, and intuitive platforms to purchase high-end products. This project addresses these needs by providing a specialised platform for luxury retail.

2.2 Motivation

The primary motivation was to bridge the gap between complex enterprise ecommerce solutions and simple static sites — creating a lightweight yet powerful system capable of handling real-world transactions and inventory.

2.3 Objectives

- Develop a secure user authentication system with role-based access.
- Implement an efficient relational database schema for managing complex relationships between users, products, and orders.
- Create a responsive and visually appealing user interface.
- Ensure data integrity during checkout and inventory updates.

2.4 Scope

The project encompasses the following modules:

- User Management (Registration, Login, Profile)
- Product Catalog (Filtering by categories, Product details)
- Shopping Cart (Add, Update, Remove items)
- Order Processing (Checkout, Order history, Status tracking)
- Admin Panel (Product CRUD, Inventory management)

2.5 Limitations

- The system currently supports only one currency (USD).
- Real-time payment gateway integration is simulated.
- Lacks a real-time shipping tracking API.

3. System Requirements & Analysis

3.1 Functional Requirements

- Users must be able to register and log in securely.
- Admins must have exclusive access to add, edit, or delete products.

- Customers must be able to add products to a persistent shopping cart.
- The system must generate order records and snapshots of prices at the time of purchase.
- A contact form must be available for user inquiries.

3.2 Non-Functional Requirements

Category	Requirement
Security	Sensitive data such as passwords must be hashed using bcrypt.
Performance	API responses optimised using database indexing and connection pooling.
Scalability	System must handle increasing numbers of products and users.
Reliability	Database transactions must guarantee ACID properties.

4. User Roles

Role	Capabilities
Admin	Manage entire product catalogue, update stock levels, oversee system data, and view all orders.
Customer (User)	Browse products, manage personal shopping cart, place orders, and view purchase history.

5. Data Dictionary

Table	Primary Key	Key Columns
users	id (PK)	name, email (Unique), password, role (ENUM), created_at
categories	id (PK)	name (Unique), description
products	id (PK)	name, description, price, stock, image_url, category_id (FK)
cart_items	id (PK)	user_id (FK), product_id (FK), quantity
orders	id (PK)	user_id (FK), total_amount, status (ENUM), created_at
order_items	id (PK)	order_id (FK), product_id (FK), quantity, price
contact_messages	id (PK)	email, phone, message, submitted_at

6. Normalization

The database is designed to comply with the **Third Normal Form (3NF)**:

Normal Form	Rule Applied
1NF	All attributes are atomic; no repeating groups.
2NF	No partial dependencies; all non-key attributes are fully dependent on the primary key.
3NF	No transitive dependencies. Product categories are stored in a separate table to avoid redundant descriptions in the products table.

7. Application Stack

Layer	Technology
Backend	Node.js with Express.js
Database	MySQL (relational database)
Authentication	JSON Web Tokens (JWT) for stateless session management
Frontend	Vanilla HTML5, CSS3, and JavaScript (ES6+)

8. Key User Interfaces

Interface	Description
Landing Page	Showcases featured products and categories.
Product Management (Admin)	Dashboard for adding and updating inventory.
Shopping Cart	Dynamic interface for managing selected items.
Checkout Page	Summarises orders and captures shipping details.
Order History	Provides users with a detailed log of past transactions.

SQL Table Creation Scripts

— users table

```
CREATE TABLE IF NOT EXISTS users (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  email VARCHAR(100) NOT NULL UNIQUE,  
  password VARCHAR(255) NOT NULL, -- hashed with bcrypt  
  role ENUM('customer', 'admin') DEFAULT 'customer',  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

— categories table

```
CREATE TABLE IF NOT EXISTS categories (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100) NOT NULL UNIQUE,  
  description TEXT  
);
```

— products table

```
CREATE TABLE IF NOT EXISTS products (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(255) NOT NULL,  
  description TEXT,  
  price DECIMAL(10, 2) NOT NULL,  
  stock INT NOT NULL DEFAULT 0,  
  image_url VARCHAR(255),  
  category_id INT,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (category_id) REFERENCES categories(id) ON DELETE SET NULL  
);
```

— cart_items table

```

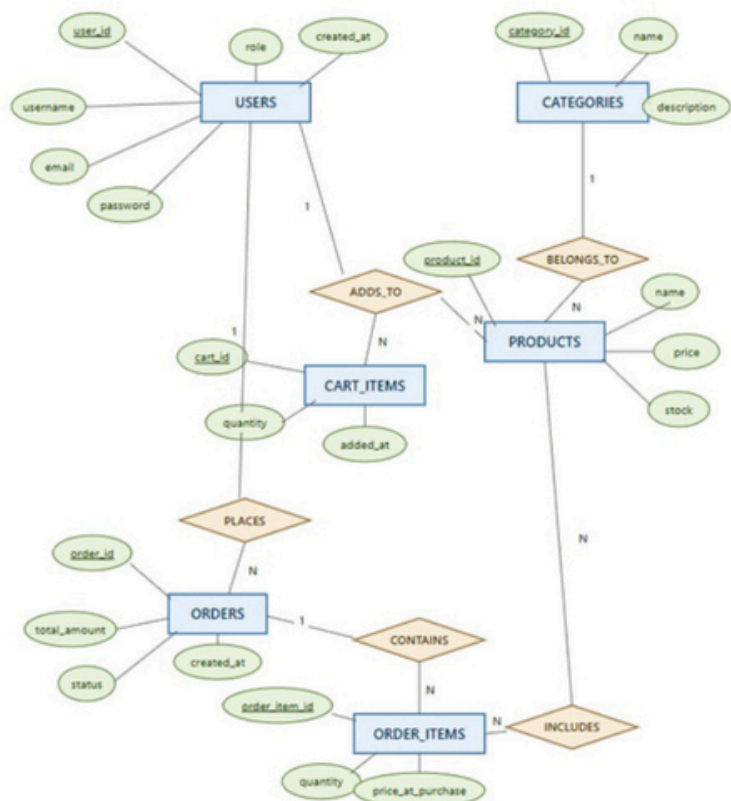
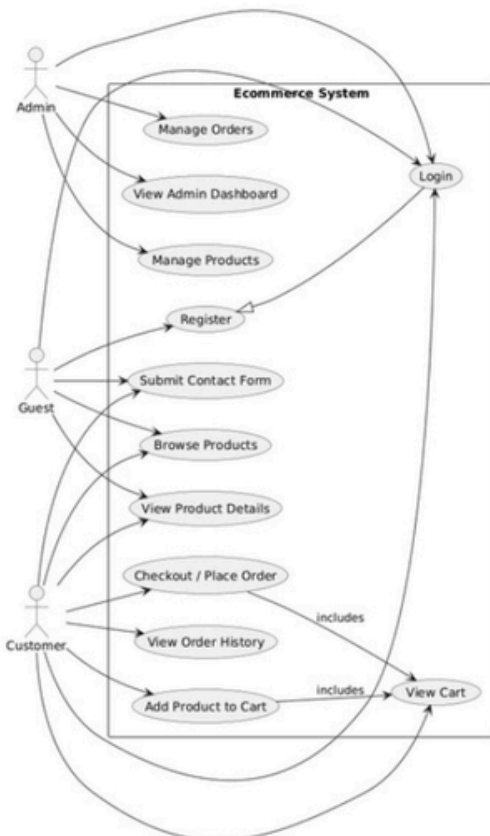
CREATE TABLE IF NOT EXISTS cart_items (
  id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL,
  product_id INT NOT NULL,
  quantity INT NOT NULL DEFAULT 1,
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
  FOREIGN KEY (product_id) REFERENCES products(id) ON DELETE CASCADE
);

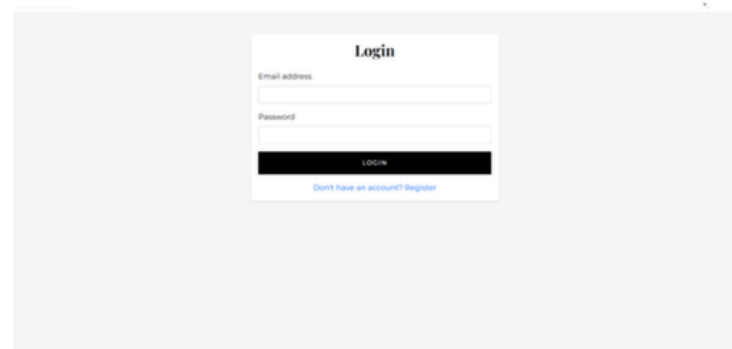
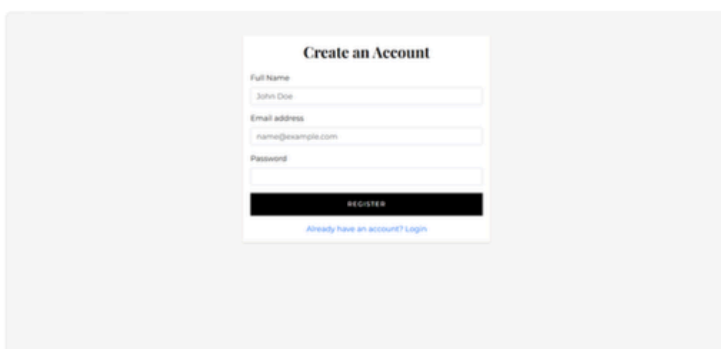
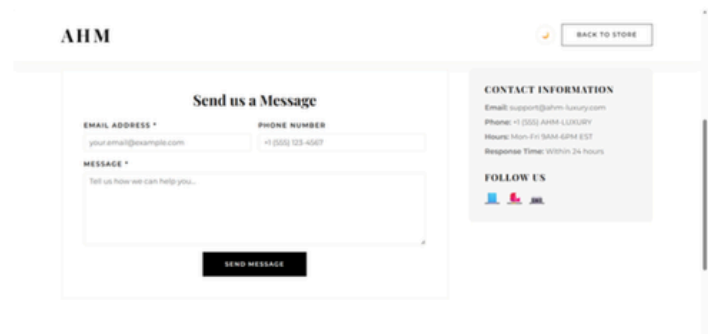
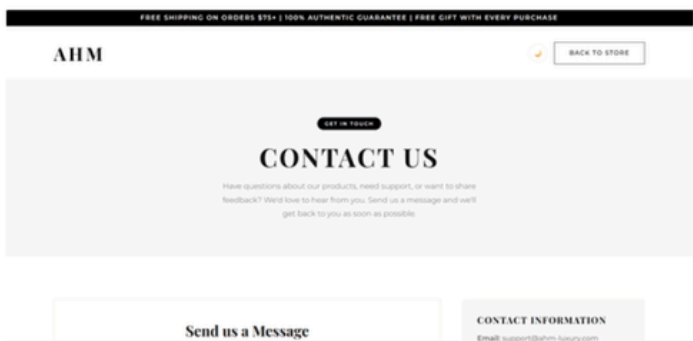
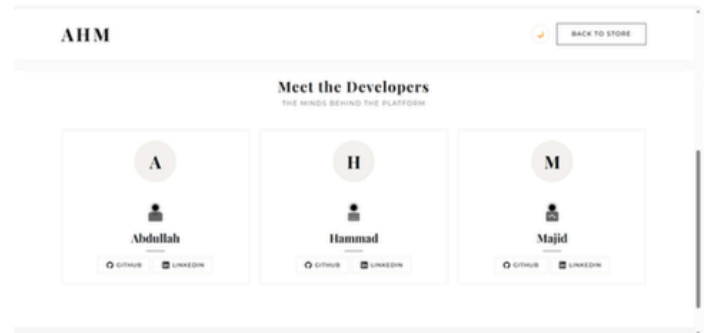
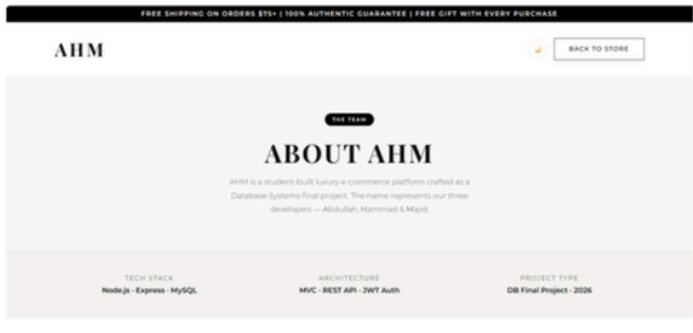
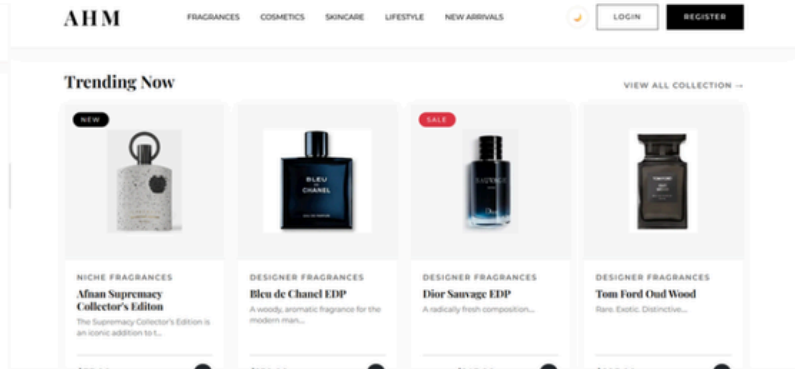
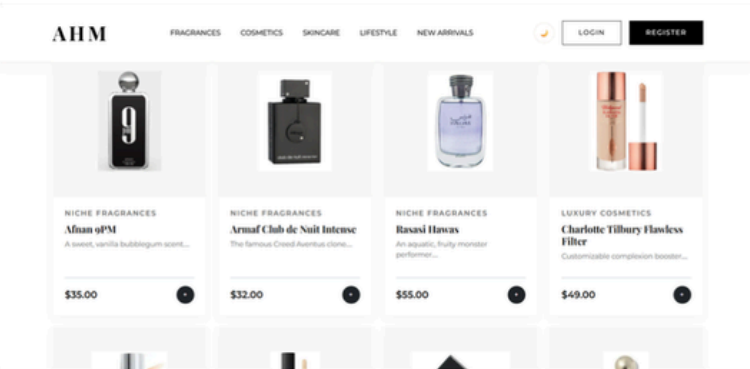
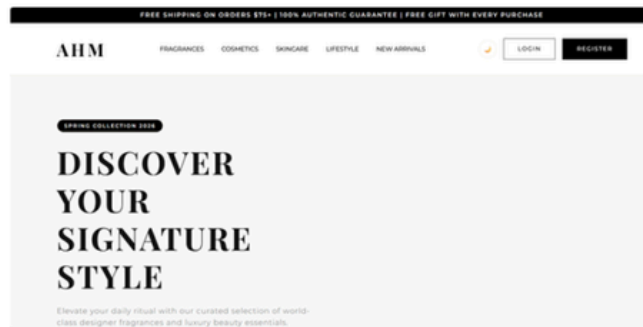
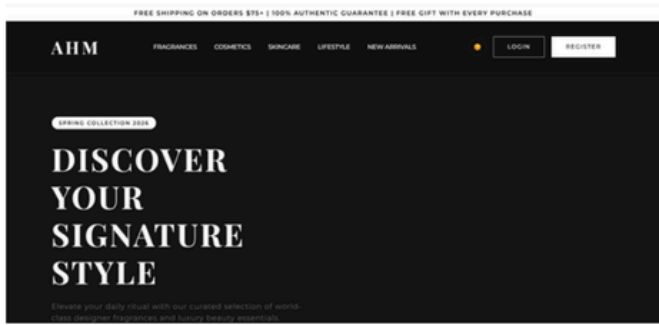
```

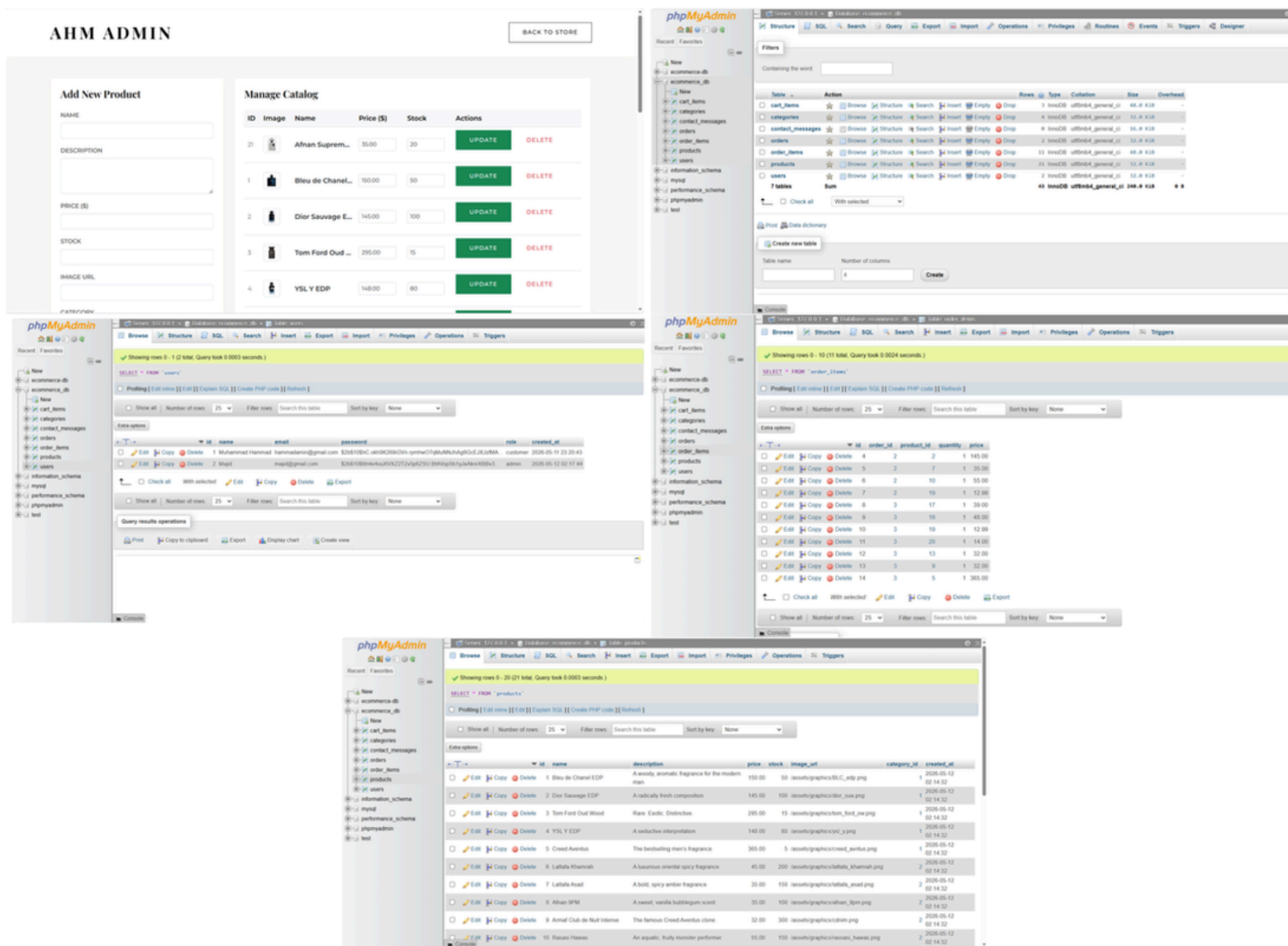
Sample Data — Products

Product Name	Category	Price	Stock
Bleu de Chanel EDP	Designer Fragrances	\$150.00	50
Dior Sauvage EDP	Designer Fragrances	\$145.00	100
Tom Ford Oud Wood	Designer Fragrances	\$295.00	15
YSL Y EDP	Designer Fragrances	\$148.00	80
Creed Aventus	Designer Fragrances	\$365.00	5
Lattafa Khamrah	Niche Fragrances	\$45.00	200
Lattafa Asad	Niche Fragrances	\$35.00	150
Afnan 9PM	Niche Fragrances	\$35.00	100
Armaf Club de Nuit Intense	Niche Fragrances	\$32.00	300
Rasasi Hawas	Niche Fragrances	\$55.00	150

USE CASE and ERD Diagram







9. Results & Discussion

9.1 Achievements

- Successfully implemented a secure, role-based ecommerce workflow.
- Achieved high database performance through efficient query design and connection pooling.
- Developed a clean, responsive UI without heavy external frameworks.

9.2 Issues & Resolutions

Challenge	Solution
Maintaining cart state across sessions.	Persisted cart items directly in the MySQL database.
Preventing stock overselling during checkout.	Implemented server-side validation in the checkout controller.

9.3 Observations

MySQL's relational structure proved ideal for maintaining the complex links between users, orders, and products, providing reliable data consistency throughout all transaction workflows.

10. Conclusion

The **AHM Ecommerce System** demonstrates a successful integration of web technologies and database management principles. It provides a scalable and secure platform ready for further expansion into a full-scale commercial application.

11. Limitations

- Restricted to single-location warehouse logic.
- No integrated customer review or rating system.
- Lacks automated email notifications for orders.

12. Future Enhancements

Enhancement	Description
Global Search & Price Comparison	Implement site-wide search with intelligent filtering and product price comparison tools.
Multi-location Support	Expand the database to handle multiple warehouses and localised inventory.
Advanced Analytics Dashboard	Integrate visualisations for sales trends and customer behaviour using charts.

13. Integration Capabilities

- **Mobile Application:** The existing RESTful API is designed to be consumed by a React Native or Flutter mobile application.
- **Currency Conversion:** Integration with third-party APIs for real-time currency conversion.
- **Shipping Calculation:** Automated shipping cost calculation via external carrier APIs.

14. Administrative Enhancements

- Implementation of a user management module to block or delete accounts.
- Automated low-stock alerts to notify admins of critical inventory levels.
- Detailed revenue reporting tools and export functionality for admins.

AHM — Built with precision. Crafted with elegance.